TiltTwin

Remote CAD teaching through physical motion
Design process report

Kevin, Ayushi and Prince
CPSC 581, Fall 2026, Group Project 2: Expertise Sharing
Source: github.com/Kevin398/CPSC581-TiltTwin | Live system: tilttwin.vercel.app

Summary. An expert on a laptop demonstrates a rotation of a CAD part. A beginner on a phone reproduces it by *physically turning the phone*, so the virtual part turns as if it were held in their hand. Both people see live angle, direction and colour feedback, and both get a ✓ when the beginner matches.
Expertise conveyed: Spatial understanding of 3D rotation, meaning which axis, which direction and how far. Novices in Blender or Fusion struggle with this because the mouse mapping (orbit vs. rotate, gimbal axes) is abstract.
Configuration: One expert (laptop), one novice (smartphone), synchronous and remote, joined by a 4-letter room code.

1. Process overview

We alternated divergent and convergent phases over five iterations. Each iteration produced an artifact (sketch set, storyboard, wireframe or prototype) that was reviewed before the next step. Kevin developed the initial sketches and baseline prototype, regularly consulting teammates Ayushi and Prince for feedback. Their feedback contributed to the team's evaluation of concepts and refinement of the interaction during prototyping.

Stage	Activity	Activity type
1. Diverge	10 interaction concepts sketched (Fig. 1)	Divergent
2. Converge	Compare against criteria; pivot from CADSense to TiltTwin (Table 1)	Convergent
3. Refine	5 refinement sketches; drop transparency (Fig. 2)	Divergent
4. Specify	Storyboard; wireframes v1 → v2 (Figs. 3–4)	Convergent
5. Prototype	v1: mouse baseline (Fig. 5); v2: phone tilt + sync (Figs. 6–8)	Building
6. Evaluate	Technical checks; real-device tests; 3-person pilot; proposed A/B (§9)	Evaluation
7. Extend	Real assembly tasks, hot/cold sound and haptics (§7)	Divergent
8. Polish	UI/UX audit, fixed join link (§8)	Convergent

Methods and tools: Individual rapid sketching (in-class "10 ideas" technique), iterative feedback from teammates, team critique, a criteria-based comparison matrix, a storyboard, low-fidelity wireframes in two versions, an interactive high-fidelity prototype (React + Three.js), automated unit tests of sensor

mapping, real-device tests, a three-participant pilot study of the phone-tilt condition, and a proposed within-subjects A/B study comparing touch/mouse dragging with physical phone rotation.

2. Divergence: ten interaction concepts

Each concept answers the same question in a different way: *how can an expert's knowledge of 3D orientation reach a novice through the body rather than through words or a shared screen?*

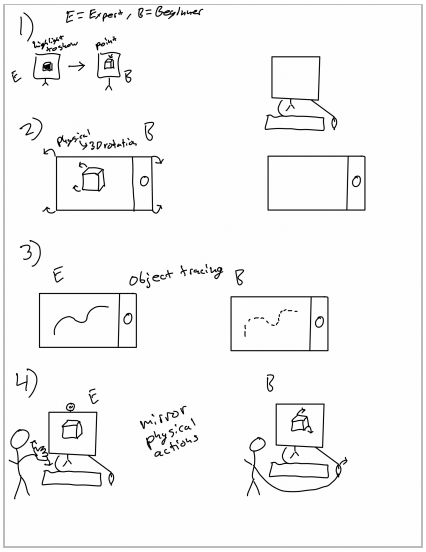


Fig. 1a Concepts 1–4: highlight and point; physical phone rotation; path tracing; mirroring the expert's hand motion (webcam).

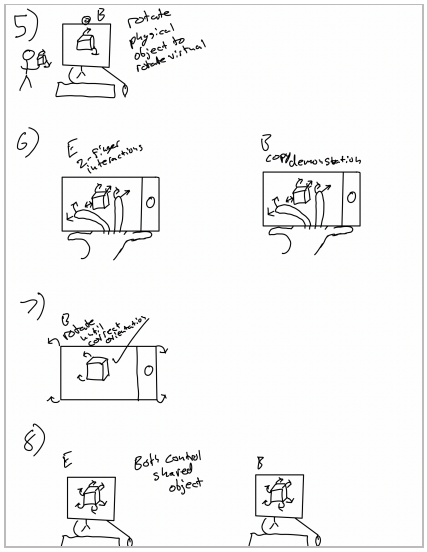


Fig. 1b Concepts 5–8: rotate a physical proxy object; copy two-finger gestures; rotate until correct; both control one shared object.

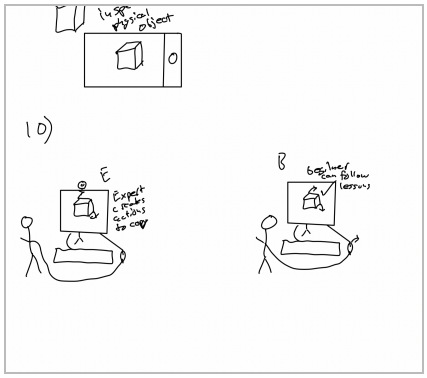


Fig. 1c Concepts 9–10: inspect a physical object beside its virtual twin; the expert records actions as lessons the beginner follows.

Collaborative sketch evaluation. Kevin produced the initial concept sketches and shared them with Ayushi and Prince for feedback. This provided a team perspective during early exploration rather than relying only on individual judgment. The concepts were then compared for embodied interaction, their ability to communicate CAD expertise, and their feasibility within the project timeline. The sketching process helped the team narrow the problem before committing to implementation.

3. Convergence: choosing the concept

Our first direction, **CADSense**, was a mouse-driven tutor covering extrude, rotate and scale. The beginner followed on-screen arrows to match a target model. When we checked it against the brief and rubric (25% of the grade is creative *embodied* input and output, and video calls or screen sharing as the core interaction are explicitly discouraged), CADSense's mouse input looked conventional. We scored the strongest concepts from Fig. 1 against criteria taken directly from the brief.

> *I think we could improve the original CADSense idea by making it more focused on embodied interaction, since that's one of the biggest grading criteria.*
>
> *Instead of having beginners use a mouse to follow arrows and match a target 3D model, we could make a version called TiltTwin, where beginners physically rotate their phone to manipulate a 3D object. […]*
>
> *It could also simplify development since we'd focus on making one rotation interaction really polished instead of implementing extrusion, rotation, and scaling.*
>
> *The main risk is getting phone motion sensors working reliably, so we'd want to test that first.*

Fig. 1d Team Discord, the pivot proposal (excerpt, transcribed). It names the two convergence criteria we then scored in Table 1 (embodiment and a polished single interaction) and the main technical risk (sensor reliability), which we tested first.

Concept	Embodied input	Embodied output	Conveys CAD expertise	Feasible in ~2 weeks	Reliable live demo	Total
CADSense (mouse + arrows, 3 operations)	1	2	3	2	3	11
#3 Object tracing (draw a path)	2	1	1	3	3	10
#4 Mirror hand motion (webcam tracking)	3	2	2	1	1	9
#5 Rotate a physical proxy object	3	2	3	1	1	10
#8 Both control a shared object	2	2	2	2	2	10
#2 + #7 + #10 → TiltTwin (phone is the part)	**3**	**3**	**3**	**2**	**2**	**13**

Table 1. Scores from 1 (weak) to 3 (strong). These are qualitative design judgments used to compare concepts against our project constraints, not measured usability results. The proxy-object idea (#5) scored high on embodiment, but it requires tracking hardware or computer vision that would be difficult to implement reliably within our timeline. A smartphone already contains orientation sensors, allowing **the phone itself to become the physical proxy**.

> **Decision.** Merge #2 (physical rotation), #7 (rotate until correct) and #10 (expert creates actions to copy) into TiltTwin. We deliberately **gave up breadth** (extrude and scale) to make one embodied interaction, rotation, complete and polished. This also enables a controlled comparison with mouse-based rotation in a future A/B study (§9).

4. Iteration 1: refining TiltTwin

> *i think we should keep all the ideas from the second set of sketches except for the transparency*

Fig. 2c Team Discord, the convergence decision on the refinement sketches in Fig. 2a–b (transcribed). The reasoning behind dropping transparency is set out below.

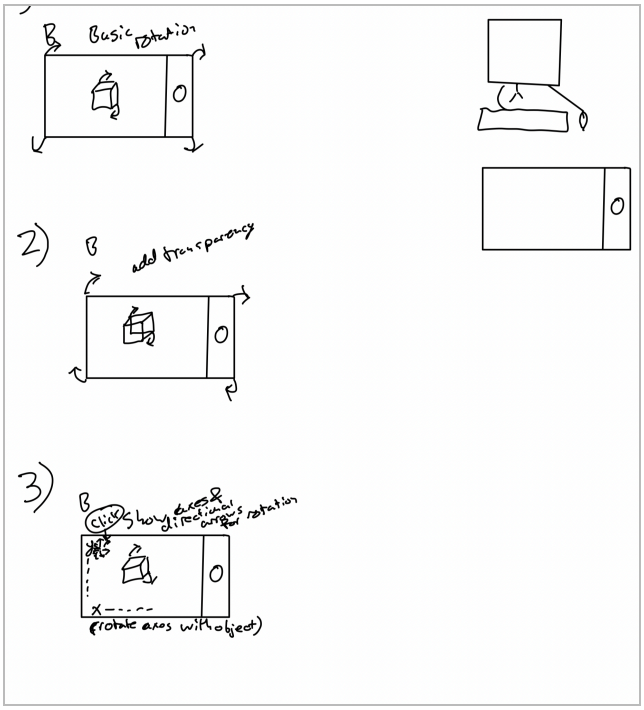


Fig. 2a (1) basic physical rotation; (2) a transparent "ghost" of the target over the part; (3) tap to show axes and direction arrows that rotate with the object.

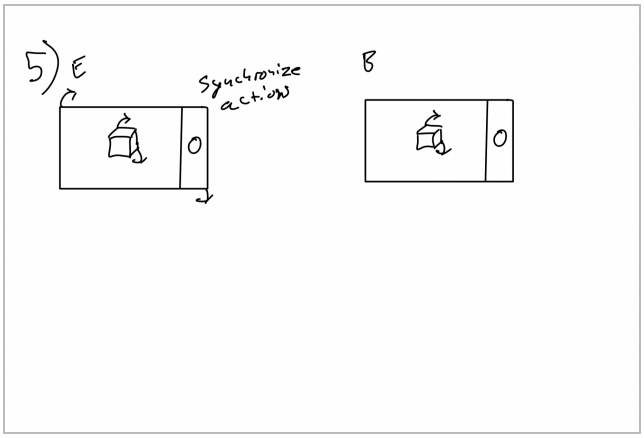


Fig. 2b (4) ✓ feedback on reaching the correct orientation; (5) synchronize the expert's and beginner's actions.

Kept: 1, 3, 4, 5. **Dropped:** 2, the transparency overlay. On a phone-sized screen, two overlapping, semi-transparent L-brackets are hard to read. It becomes unclear which edges belong to which part, and when the part is close to the target the two shapes merge. The overlay also invites "matching shapes by eye" rather than understanding *which axis and which direction*, which is the expertise we want to convey. We replaced it with three cues that each answer one question:

- **Where should it end up?** A small window showing the expert's *demonstration*: the part animating from 0° to the target on a loop.
- **Which way do I turn?** A coloured arc arrow around the highlighted axis, from the current angle to the target.
- **How far am I?** A large live gauge ("22° / 45°") plus colour coding: orange when far, yellow when close, green when matched.

Kevin shared the refinement sketches with Ayushi and Prince for feedback during this design stage. The final direction prioritized readable mobile visuals and explicit guidance about rotation direction and angle, rather than overlapping transparent models.

5. Iteration 2: storyboard and wireframes

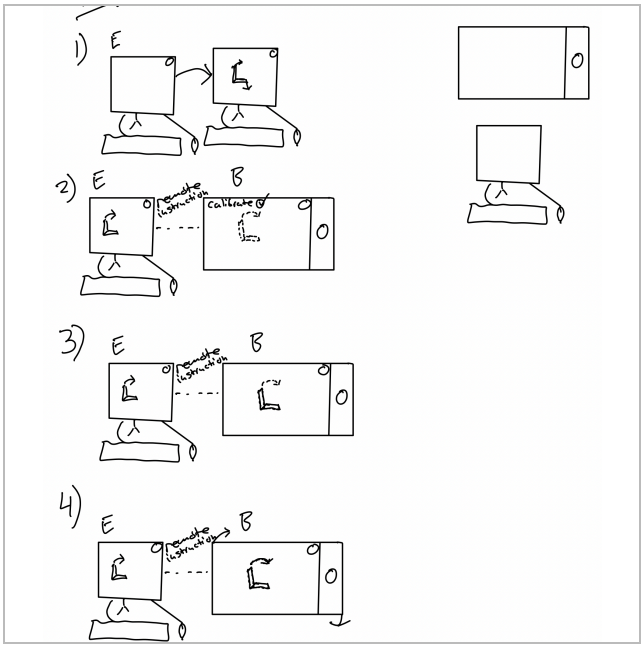


Fig. 3a Storyboard 1–4. The expert sends a remote instruction (1–2); the beginner calibrates (2); the beginner rotates the phone and the part follows (3–4).

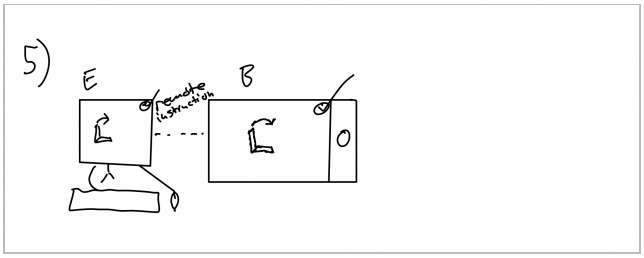


Fig. 3b Storyboard 5. Both screens show the completion check once the beginner matches the expert's rotation.

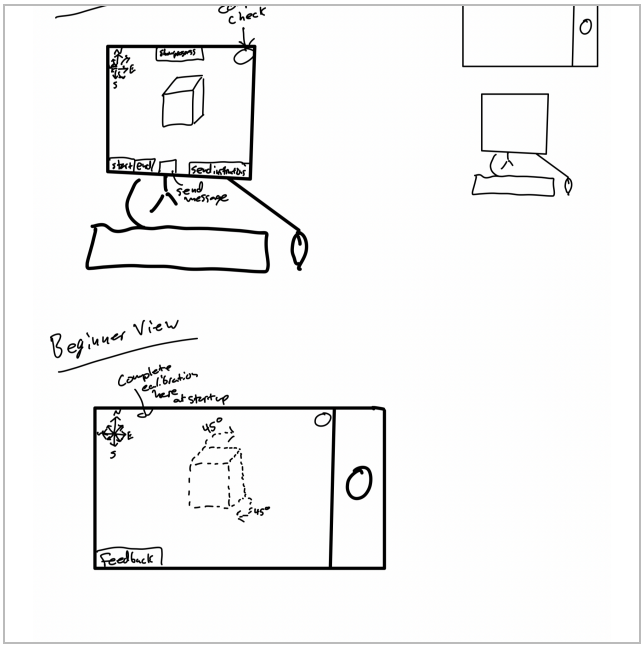


Fig. 4a Wireframes v1. The expert has a "show progress" button, start/end, send instruction/message and a completion check. The beginner sees a dashed target with 45° labels, a compass, and a feedback button.

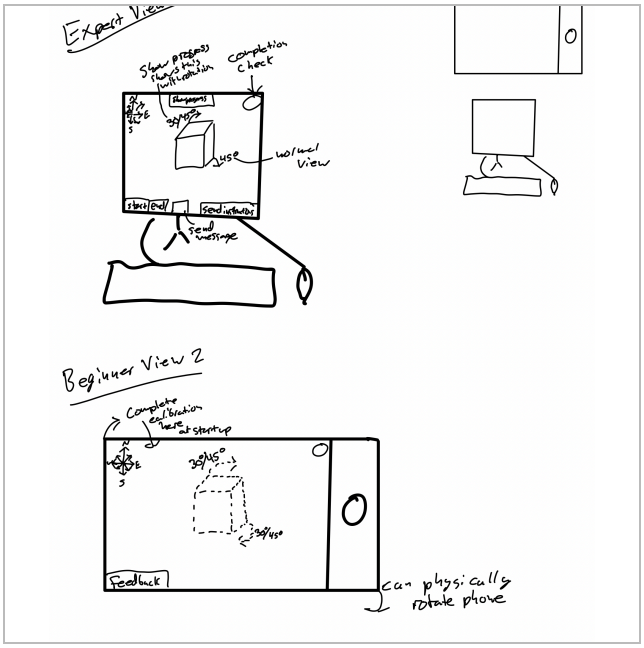


Fig. 4b Wireframes v2. The expert sees the beginner's progress as a live angle ("30°/45°") on the model. The beginner calibrates at startup, sees the same angle labels, and physically rotates the phone.

v1 → v2 changes (Fig. 4a → 4b): the progress indicator changed from a vague "show progress" button to a **live angle on the model itself**, shown to *both* people. Showing "30°/45°" to both gives the expert a precise thing to correct ("15° more") and gives the novice an exact goal. v2 also makes **physically rotating the phone** the explicit input, replacing on-screen controls.

6. Iteration 3: prototypes

Prototype v1: mouse baseline (CADSense-style, "Version A")

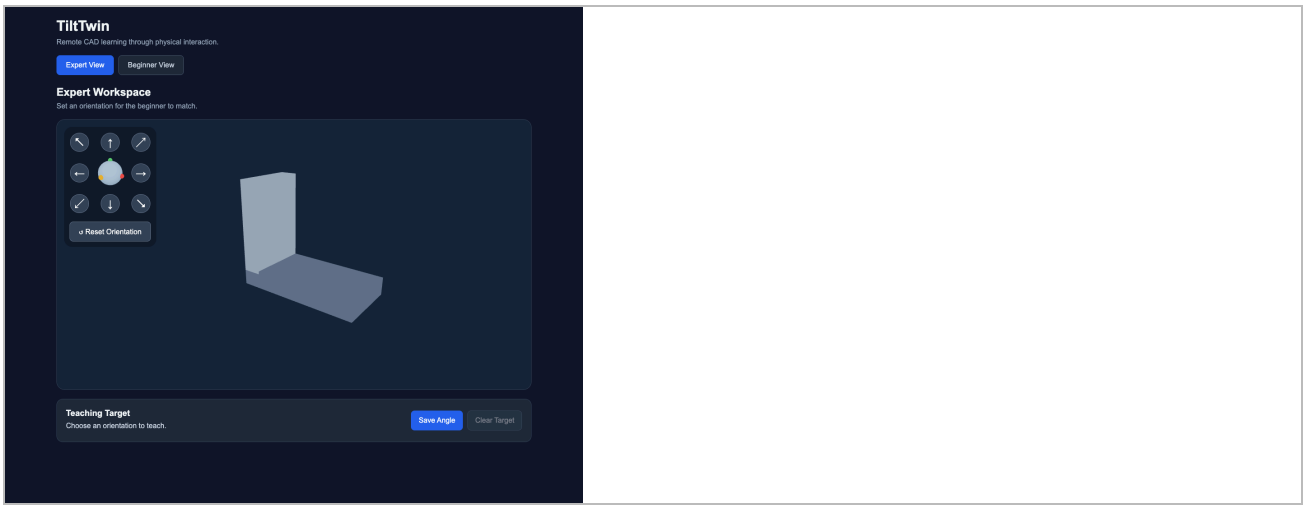


Fig. 5 v1 expert workspace. Free 2-axis rotation by mouse drag or an 8-way arrow pad, plus "Save Angle". The beginner pressed "Check Angle" for feedback. Both roles ran in one browser tab.

What v1 taught us. Free 2-axis rotation made instructions hard to express ("rotate it like… this"). Feedback only came when the beginner pressed a button. Running in one tab meant there was no remote collaboration yet. These became the requirements for v2: **one axis per lesson**, **continuous feedback** and **real two-device sync**.

Collaborative prototyping. Kevin implemented the initial browser-based prototype and shared the evolving interaction with Ayushi and Prince for feedback. The prototype gave the team something concrete to evaluate beyond sketches and wireframes. This process helped identify the limitations of free-form mouse rotation, delayed angle checking, and the absence of communication between separate devices. The next iteration focused on clearer lessons, continuous feedback, and real-time synchronization.

Prototype v2: TiltTwin (final system)

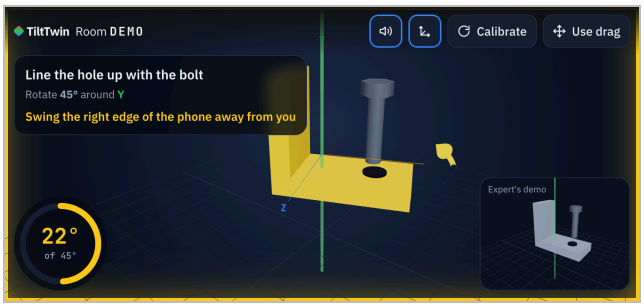


Fig. 6a Beginner, mid-rotation. The instruction gives a body-centred hint ("Swing the right edge of the phone away from you"). The Y axis is highlighted, the arc arrow shows the remaining turn, the gauge reads 22°/45°, and everything is yellow because the part is close. The expert's looping demo plays bottom-right.

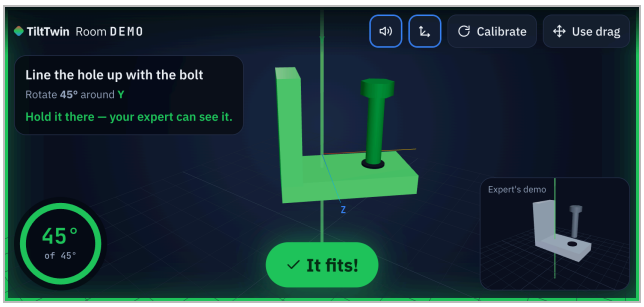


Fig. 6b After remaining within 8° of the target for 0.8 seconds, the part, frame and gauge turn green, and a matching confirmation appears. An audio chime provides feedback, with haptic feedback attempted where supported.

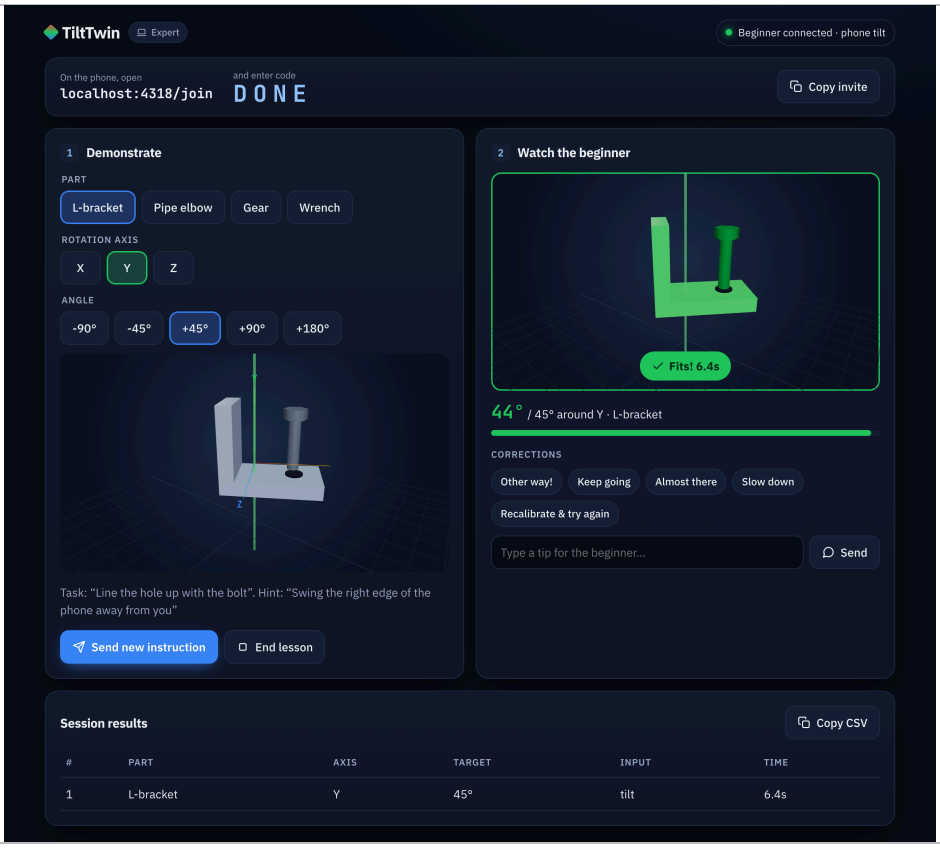


Fig. 7 Expert view. Left: choose an axis and angle, preview the demonstration loop, and send. Right: a live mirror of the beginner's part, angle readout, quick corrections and a free-text tip. Bottom: the session log records each lesson's input type and completion time (designed to support later usability testing).

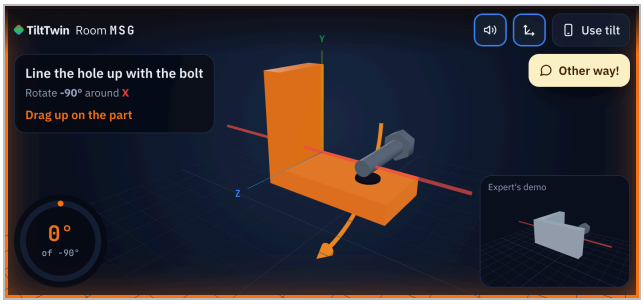


Fig. 8a The expert's correction ("Other way!") arrives as a toast. A –90° X lesson shows the arc arrow for the full remaining turn.

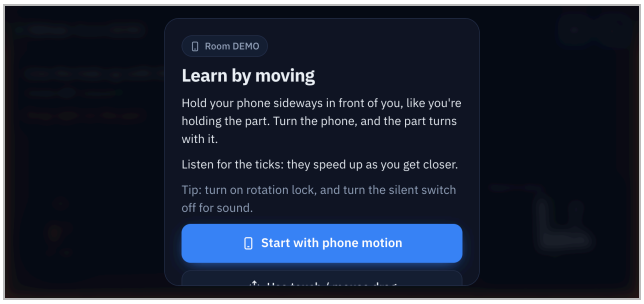


Fig. 8b Onboarding: how to hold the phone, a rotation-lock tip, and a drag fallback for devices without motion sensors.

Key design decisions in v2

Decision	Why	Alternatives considered
The phone is the part. Turning the phone turns the part the same way, measured around the lesson's axis.	The most direct embodied mapping: the hand learns the rotation, not just the eye.	On-screen arrows (v1); full free 3-DoF rotation (too hard to judge against one target).
One axis per lesson (swing-twist decomposition isolates rotation around X, Y or Z).	Hands wobble. Ignoring off-axis wobble keeps feedback stable and matches how CAD tools teach rotation about one axis.	Matching full orientation (frustrating jitter); Euler angles (gimbal lock near an upright phone).
Calibration: the current pose becomes 0° when a lesson arrives, and a Calibrate button is always available.	Everyone holds the phone differently. A relative zero avoids compass drift and removes "hold it exactly like this" instructions.	Absolute compass heading (drifts indoors; differs between phones).
Body-centred hints ("Tilt the top toward you", "Swing the right edge away from you").	Translates CAD axis language into motor language. Signs are verified by unit tests against the sensor specification.	Showing only X/Y/Z and ± (abstract for novices).
8° tolerance, 0.8 s hold.	Passing briefly through the target should not count as successful completion. The tolerance allows small variations in hand positioning, while the hold requirement encourages deliberate control of the rotation.	The v1 "Check Angle" button provided delayed feedback. A stricter 5° tolerance was also considered. The selected thresholds were retained for the prototype, with further usability testing needed to evaluate their suitability.
Feedback in four channels: colour, number, arrow, sound/vibration.	The beginner's eyes move between the part and the phone's orientation, so redundant cues help. Vibration confirms success without looking.	Transparency overlay (dropped, §4).
Quick corrections plus free text for the expert.	The expert watches a live mirror and must correct within seconds; one-tap phrases are faster than typing.	Voice or video call (discouraged by the brief; doesn't use the shared model).
Firebase Realtime Database rooms with a 4-letter code.	Sub-second sync without our own server. Codes avoid confusable characters (no 0/O, 1/I).	WebRTC peer-to-peer (fails behind some NATs; risky for the live demo).

7. Iteration 4: from "match an angle" to "make it fit"

After implementing the core phone-rotation interaction, we explored how to make TiltTwin **more useful in real CAD work** and **more engaging**, without weakening the embodied interaction. We compared several possible additions and selected features that strengthened the learning experience while remaining feasible within the project timeline:

Candidate	Value	Cost or risk	Decision
Part library	Beginners practise on parts they will actually model	Low	Built
Assembly "fit" task with a mating part	Gives the rotation a purpose: in CAD assemblies you orient a part so it mates with another	Medium	Built
Hot/cold vibration	Feel the answer without looking: strong embodied output	iPhone Safari has no vibration API	Built, with sound as the guaranteed channel
Stars and streaks	Fun and motivating	Low, but weak link to expertise sharing	Deferred

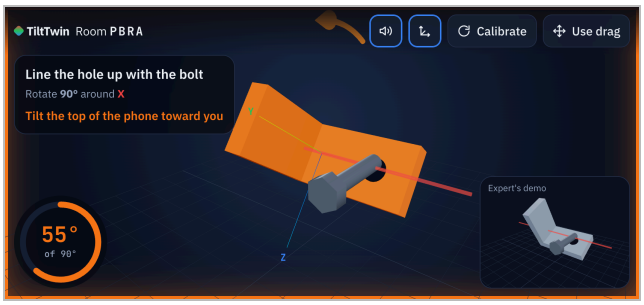


Fig. 9a L-bracket: line its hole up with the bolt (X axis).



Fig. 9b Pipe elbow: connect its outlet to the pipe (Y axis).

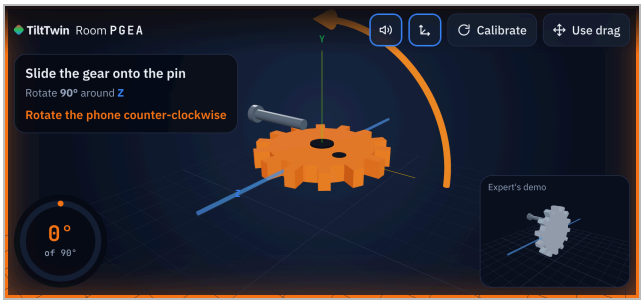


Fig. 9c Gear: slide its pin hole onto the pin (Z axis).

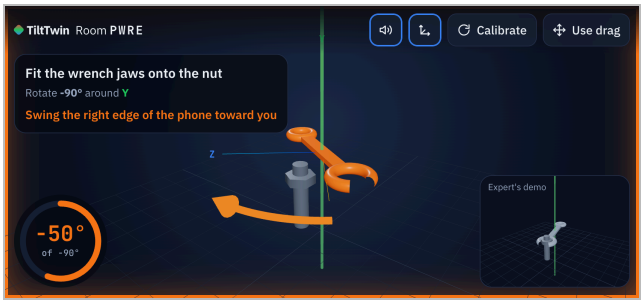


Fig. 9d Wrench: fit the jaws onto the nut (–90° about Y). The arrow shows a clockwise turn.

- **Mating parts, not a ghost.** Each part has a *feature* (a hole, an outlet or the jaws), and a solid mating part (a bolt, pipe, pin or nut) is fixed where that feature will be at the target. This keeps the decision from §4: no transparent overlay. It also turns an abstract angle into a physical goal, and the moment of success becomes "it clicks into place". A unit test checks that no feature sits on an X/Y/Z axis line; otherwise rotating about that axis would never visibly move it, and the task would be impossible to see.
- **Hot/cold guidance.** While a lesson is active, the phone ticks. The ticks speed up (from ~1 to ~9 per second) and rise in pitch as the part gets closer, and stop when it fits. On Android each tick is also a vibration pulse. On iPhone we use the light haptic that iOS 18+ gives when a native switch control is toggled, because Safari has no vibration API. That trick is undocumented, so **sound is the primary fallback where audio playback is permitted**. The beginner can find the target by ear and by feel while their eyes are on the part. A sound button turns it off.
- **Expert side.** The expert chooses the part as well as the axis and angle. The demonstration loop and live mirror both display the mating part. The session log records the selected part, allowing a future

A/B usability study to compare interaction performance across different CAD components.

8. Iteration 5: usability and visual audit

Before the critique we audited the whole interface against a professional UI/UX checklist (the *UI/UX Pro Max* guideline database: accessibility, touch, typography, motion and forms). We also simplified how beginners join, after noticing that a new long link per lesson is awkward to send to a phone.

Finding	Guideline	Change
A new long link for every lesson had to be sent to the phone	Predictable navigation; fewer steps	One fixed link (`tilttwin.vercel.app/join`, easy to bookmark) plus a 4-letter code shown large on the expert's screen. The code is checked against live rooms, and a mistyped code gets an inline error.
Emoji used as icons (🔊 💬)	One consistent SVG icon family	Inline stroke icon set; icon-only buttons have accessible names
No visible keyboard focus on dark buttons	Focus states (high severity)	`:focus-visible` ring on every control
Phone toolbar buttons were 38 px	Touch targets ≥ 44 px	All controls ≥ 44 px
Code field used a placeholder as its label	Visible labels; errors next to the field	Visible label, helper text and inline error
Thin bar meter for progress	Glanceable feedback	Radial progress ring around the live angle, in monospaced numbers that don't jitter
Home page didn't show the product	Show the core interaction first	Interactive 3D hero (pick a part, watch it fit) and a three-step "How it works"
Success was a static badge	Motion should convey meaning	A pulse ring for the "click into place" moment; all motion is disabled under reduced-motion

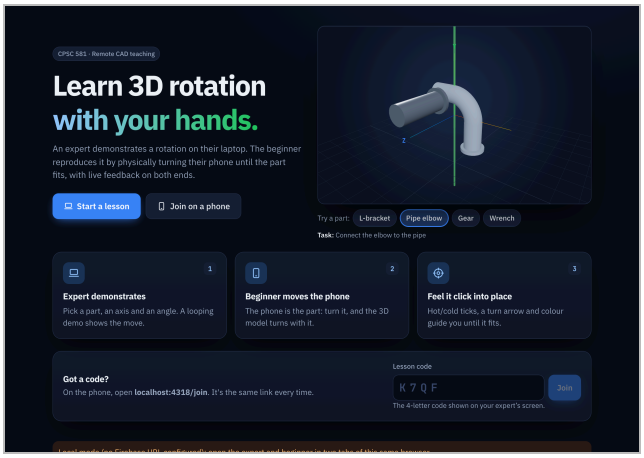


Fig. 10a Home: interactive 3D preview with part chips, "How it works", and code entry.

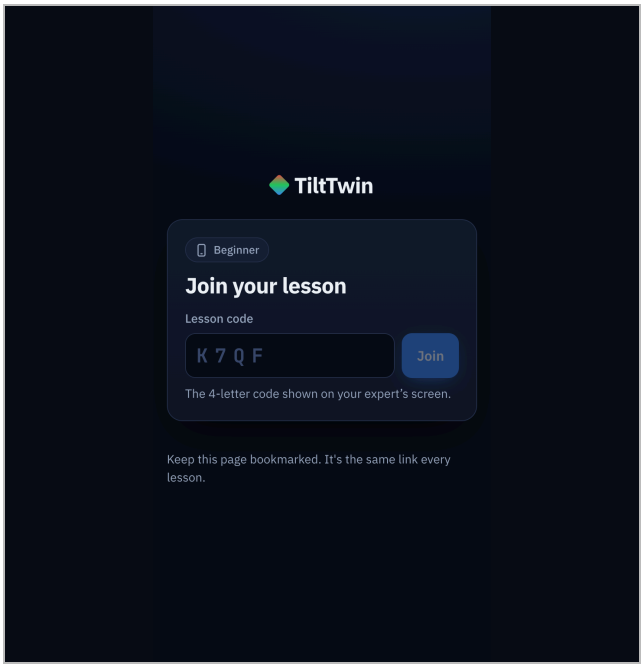


Fig. 10b The fixed join page on a phone. Typography is IBM Plex Sans with JetBrains Mono for codes and angles.

9. Evaluation

9.1 Technical verification (completed)

- **Automated sensor-mapping tests.** The project includes tests for angle wrap-around at ±180°, quaternion twist extraction, and the mapping between physical phone movement and virtual CAD rotation. The tests cover portrait and both landscape phone orientations, checking the expected direction of rotation along X, Y and Z. Additional tests verify that the assembly features move visibly around each rotation axis. These tests helped validate the mathematical foundation of the interaction.
- **Simulated sensor testing.** During development, simulated `deviceorientation` events were used to exercise the beginner's rotation controls and feedback states. The expert's synchronized view was also tested using separate browser tabs.
- **Bug identified and corrected.** An earlier prototype had an issue where repeated synchronization updates restarted the completion timer, preventing the matched state from activating consistently. The timer was updated to use the lesson ID rather than the changing lesson object.
- **Real-iPhone testing — October 20, 2026.** Prince ran the deployed app with the expert view on a second device and an iPhone as the beginner, using phone motion. The expert's session log recorded **X +90° completed in 8.1 s** and **Z +45° in 4.2 s**. On the **Y** lesson, however, Prince moved the phone the opposite way from what the hint intended (see the test-driven change in §9.2). After the fix, a re-test confirmed that **X, Y and Z all moved in the expected direction**. A second session that day tested the four parts and the hot/cold guidance on the iPhone: each part fitted its mating part when matched, and the hot/cold sound worked.
- **Real-device testing — October 21, 2026.** Kevin tested the deployed application with an iPad acting as the expert and a smartphone acting as the beginner. Both devices used Safari. The devices successfully connected through a shared lesson session. The beginner used physical phone-motion controls to rotate

the virtual CAD part, and the observed movements behaved in the expected directions. Communication between the expert and beginner devices worked during the test.

> **Verification outcome.** The real-device walkthrough demonstrated that TiltTwin's core expertise-sharing interaction works beyond a simulated browser environment. An expert can deliver a CAD rotation task remotely, while the beginner manipulates a virtual model through physical phone movement. This establishes functional feasibility on the tested devices, but does not by itself establish learning effectiveness or compatibility with every smartphone and browser.

9.2 Pilot study and proposed A/B comparison

Completed exploratory evaluation. Our initial hands-on evaluation focused on whether TiltTwin's central interaction could be performed successfully using two separate devices. Two real-device sessions (expert device + iPhone on Oct 20; iPad + smartphone in Safari on Oct 21) confirmed session communication and the expected physical-to-virtual rotation behaviour, and the first session produced a design change (below). These were team walkthroughs, so we followed them with a small pilot study with three participants.

Pilot study: phone-tilt condition (n = 3). Three participants (P1–P3) each joined a separate lesson session on their phone (codes FR2A, 89ZM and H2UM), then completed three rotation tasks by physically turning the phone. A team member acted as the expert, sending each task from the expert view and watching the live mirror. Tasks covered all four parts, all three axes, and positive and negative angles of 45° and 90°. Completion time is recorded automatically by the expert's session log, from the moment the instruction arrives to the moment the part is first held on target (the 0.8 s hold is excluded).

	Task 1	Task 2	Task 3	Mean
P1	Elbow, Y +45°: 6.5 s	Gear, Y +45°: 1.8 s	Wrench, Z –45°: 4.0 s	4.1 s
P2	Bracket, X +90°: 5.1 s	Gear, X +90°: 2.7 s	Gear, Z +90°: 4.0 s	3.9 s
P3	Bracket, Y –90°: 3.1 s	Bracket, Z –90°: 3.7 s	Elbow, Z –90°: 3.4 s	3.4 s

Table 2. Pilot results, phone-tilt condition. All 9 tasks completed; mean 3.8 s (SD 1.4), median 3.7 s, range 1.8–6.5 s.

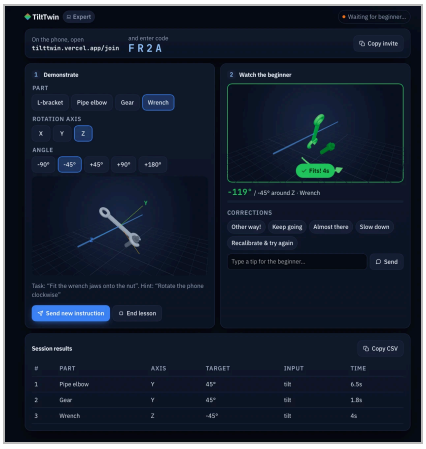


Fig. 11a P1's session log on the expert screen.

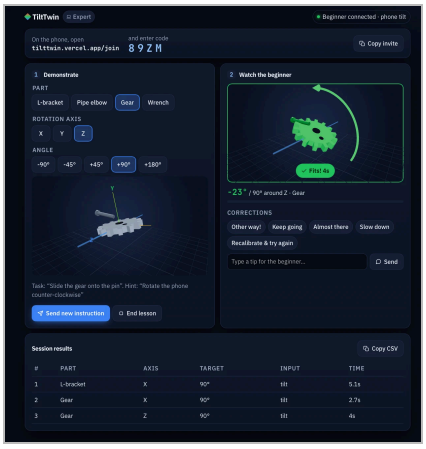


Fig. 11b P2's session log.

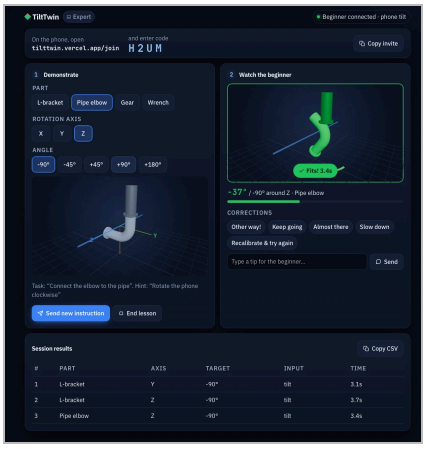


Fig. 11c P3's session log.

Findings.

- **Every task was completed**, on every axis, including the Y axis whose hint had been misread before it was reworded (see the box below). Mean times were similar across axes (X 3.9 s, Y 3.8 s, Z 3.8 s), so no axis stood out as harder.
- **Possible warm-up effect.** The first task was the slowest for P1 and P2 (6.5 s and 5.1 s). Their later tasks averaged 3.1 s. Three participants is too few to separate learning from task difficulty.
- **Unexpected: the part keeps moving after "Fits!".** In all three sessions, the expert's live readout had drifted far from the target *after* the success message: −119° on a −45° task, −23° on a +90° task, −37° on a −90° task (visible in Fig. 11a–c). The likely cause is that participants relaxed or lowered the phone once the task felt finished, and the mirror kept following the phone. For the expert this looks like an error next to a ✓. **Change made after the pilot:** once the part fits, it now stays snapped to the target pose on both screens (the beginner's view and the expert's mirror) until the next instruction arrives. We verified this by fitting a part and then turning the phone 150° away: both screens stayed at the target, and the next instruction released the part back to live tracking.

Proposed A/B comparison (next step). The pilot measured only the phone-tilt condition. To evaluate whether physical phone rotation helps beginners understand spatial manipulation, we designed a within-subjects comparison between two input methods:

- **Condition A — Touch/mouse dragging:** The beginner rotates the CAD part using the application's drag controls.
- **Condition B — Physical phone rotation:** The beginner completes equivalent tasks by physically rotating a smartphone.

Participants would complete comparable rotation tasks in both conditions, such as Y +45°, X −90° and Z +90°. The order would be counterbalanced to reduce learning effects. An expert would send the instructions and observe each participant's progress.

Measure	Planned collection method	Purpose
Completion time	Session log for each completed task	Compare task efficiency
Wrong-direction starts	Observer records initial movement errors	Evaluate directional understanding
Expert corrections	Count corrections sent during each task	Estimate guidance requirements
Axis understanding	Participant rating from 1 to 5	Evaluate perceived spatial understanding
Preferred input method	Participant preference and short comments	Compare subjective experience

Test-driven change: the Y-axis hint. This is the clearest case of real-device testing changing the design.

Step	What happened
Observed	On the iPhone, X and Z lessons were completed by following the hints (8.1 s, 4.2 s), but on Y the tester moved the phone the **opposite** way to the one intended.
Diagnosed	We first suspected the sensor maths. We simulated the phone held in portrait and in both landscape directions, made each hint's motion, and checked the sign on every axis. The maths was correct in all three holds (now automated tests). The problem was the wording: "*Turn the phone to the left*" can mean the phone turning left (its screen ends up facing right) or the screen turning to face left. The X and Z hints ("tilt the top toward you", "counter-clockwise") can only be read one way, which explains why only Y failed.
Changed	The hint now names a specific edge of the phone: "*Swing the right edge of the phone away from you*" (and "toward you" for negative angles).
Re-tested	Same iPhone, same day: X, Y and Z all moved in the expected direction.

Lesson: for embodied instructions, describe the motion by a physical landmark on the device, not by a direction word that depends on the reader's frame of reference.

Evaluation limitations. The pilot is small (n = 3), has no drag condition to compare against, and did not record wrong-direction starts, ratings or preferences. It shows that beginners can complete TiltTwin's rotation tasks quickly, in about 4 s each, on every axis. It does **not** show that phone tilt is faster, easier or better for learning than mouse input; that is what the proposed A/B comparison would test.

10. Known limitations and incomplete functionality

- **One axis per lesson** (by design). Compound rotations would be a sequence of lessons.
- **Rotation only**: no extrusion or scaling, and no import of real Blender or Fusion files. There are four predefined parts (bracket, pipe elbow, gear, wrench).
- **iPhone haptics** rely on an undocumented iOS 18+ behaviour and may not fire on older iOS. Hot/cold **sound** always works, but is muted when the iPhone's silent switch is on.
- **One expert and one beginner per room.** A second beginner would overwrite the first one's progress.
- **Y-axis drift**: turning left/right relies on the gyroscope and compass and can drift slowly on some phones. Press Calibrate to fix it.
- **Screen auto-rotation**: large Z-axis turns can make the phone flip its layout unless rotation lock is on. The mapping stays correct, but the HUD jumps.
- **Security**: rooms are open to anyone with the code, and the database rules only validate shape. This is acceptable for a class demo, not for production.
- The expert's session log lives in that browser tab and is lost on reload; use "Copy CSV" to keep it.

Future work: an embodied expert. In the current system only the beginner's side is embodied. The expert sets up a lesson by choosing a part, an axis and an angle on screen, and the demonstration they send is an animation generated from those choices, not their own movement. This is the main gap between TiltTwin and the brief's goal of sharing expertise *through* embodied input on both sides. We chose to make the beginner's loop reliable first, and fixed, typed-in targets also kept the pilot tasks identical across participants. The next version would let the expert open TiltTwin on their own phone and

demonstrate by physically turning it. The same orientation pipeline would capture the axis and angle of that movement, and the beginner would see the expert's real motion replayed as the demonstration before copying it. The interaction would become "show me with your hands, then I copy with mine", and it would let an expert convey how a rotation is performed (speed, hesitation, overshoot), not just where it ends.

11. Credits and AI use

Team contributions. Initial sketches, storyboard and wireframes: Kevin, with feedback from Ayushi and Prince. Baseline prototype: Kevin, with teammate feedback during prototyping and refinement. Implementation of assigned GitHub issues: Ayushi and Prince. Video presentation: Ayushi. Final deliverable: Kevin, Ayushi and Prince.

Technologies. Built with React, TypeScript, Three.js (react-three-fiber and drei), Firebase Realtime Database and Vite.

Claude (Anthropic) assistance. Claude was used to implement parts of the v2 prototype, including the phone-orientation hook, synchronization layer and feedback UI. It also assisted with unit tests, screenshots and drafting the design-process document.

ChatGPT assistance: Project planning and GitHub issues

ChatGPT (OpenAI) was used to help organize the implementation of TiltTwin into ten GitHub issues. This included breaking the project into manageable tasks, identifying development priorities and dependencies, and drafting issue descriptions for the web application, phone orientation tracking, visual guidance, expert controls, real-time synchronization, correctness feedback, design documentation, usability testing, final PDF and deployment.

> **Reconstructed example prompt (illustrative, not a verbatim transcript):**
> Help me organize our CPSC 581 group project, TiltTwin, into GitHub issues. The application lets an expert teach CAD rotations to a beginner using physical smartphone motion. Break the implementation into manageable tasks with clear descriptions, priorities, labels and acceptance criteria. Include the frontend, phone sensors, visual feedback, real-time communication, documentation, testing and final deliverables.

The generated task structure was reviewed and used to guide development. ChatGPT also assisted with frontend implementation guidance and debugging during the baseline prototype. All AI-generated suggestions and code were subject to team review. Project design decisions, implementation choices, testing and the final submitted work remained the responsibility of the team.

ChatGPT assistance: Final design-process deliverable

ChatGPT (OpenAI) was also used to assist with reviewing, editing and refining the final TiltTwin design-process report. This included identifying incomplete sections, improving written explanations, and suggesting revisions to the technical evaluation, usability testing, design iterations and team contributions.

ChatGPT helped document the successful real-device test using an iPad as the expert and a smartphone as the beginner, both running Safari. It also helped distinguish completed testing from proposed usability studies, avoiding unsupported claims or invented results.

During the final editing process, ChatGPT provided copy-and-paste HTML revisions designed to preserve the report's existing styling, figures, tables and section structure. These revisions also documented how Kevin incorporated feedback from Ayushi and Prince during initial sketching and prototyping.

> **Reconstructed example prompt 1 — Reviewing the final design-process report (illustrative, not a verbatim transcript):**
> Review our existing CPSC 581 TiltTwin design-process report and identify sections that should be improved before final submission. Help me strengthen the technical verification and usability evaluation sections using our actual testing results. We tested TiltTwin with an iPad as the expert and a smartphone as the beginner, both using Safari, and the physical motion controls behaved as expected. Clearly distinguish completed tests from proposed usability studies. Do not invent participant numbers, completion times or survey results.

> **Reconstructed example prompt 2 — Editing HTML and documenting design feedback (illustrative, not a verbatim transcript):**
> Help me update the HTML source of our TiltTwin design-process report while preserving its existing formatting, images, tables and layout. Provide complete, copy-and-paste HTML sections for the necessary revisions. Explain how I created the initial sketches and baseline prototype while regularly consulting my teammates, Ayushi and Prince, for feedback during design exploration and prototyping. Also improve the credits and AI-use disclosure to accurately acknowledge assistance from ChatGPT and Claude. Explain each proposed change clearly so I can review and apply the edits myself without automatically modifying our GitHub repository.

Review and responsibility. ChatGPT's contributions were limited to writing, editing assistance, documentation review and suggested code changes. The team reviewed the suggestions and remained responsible for the accuracy of the report, design decisions, implementation, testing and final submission. The reconstructed prompts above illustrate the types of requests made during development and documentation. They are not presented as exact historical transcripts.